

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

Uday Shankar Y (1BM24CS427)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **Uday Shankar Y(1BM24CS427)**, who is a bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Dr. RAGHAVENDRA C K Associate Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	26/10/2025	Genetic Algorithm for Optimization Problems	4 - 8
2	13/10/2025	Particle Swarm Optimization for Function Optimization	9 - 12
3	10/10/2025	Ant Colony Optimization for the Traveling Salesman Problem	13 - 17
4	17/10/2025	Cuckoo Search (CS)	18 - 21
5	17/10/2025	Grey Wolf Optimizer (GWO):	22 - 25
6	7/11/2025	Parallel Cellular Algorithms and Programs	26 - 28
7	26/10/2025	Optimization via Gene Expression Algorithms	28 - 35

Github Link: <https://github.com/UdayShankarY/BIS>

Program 1 : Genetic Algorithm for Optimization Problems:

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import random
import math

def fitness_function(x):

    return x * math.sin(10 * math.pi * x) + 1

POPULATION_SIZE = 20
MUTATION_RATE = 0.1
CROSSOVER_RATE = 0.8
GENERATIONS = 100
X_BOUND = (-1, 2) # Search space for x

def create_individual():
    return random.uniform(*X_BOUND)

def create_population(size):
    return [create_individual() for _ in range(size)]

def evaluate_population(population):
    return [fitness_function(ind) for ind in population]

def select(population, fitnesses):
    total_fitness = sum(fitnesses)
    probs = [f / total_fitness for f in fitnesses]
    return random.choices(population, weights=probs, k=2)

def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        alpha = random.random()
        child1 = alpha * parent1 + (1 - alpha) * parent2
        child2 = alpha * parent2 + (1 - alpha) * parent1
        return child1, child2
    else:
        return parent1, parent2

def mutate(individual):
    if random.random() < MUTATION_RATE:
        mutation = random.uniform(-0.1, 0.1)
```

```

        individual += mutation
        individual = max(min(individual, X_BOUND[1]), X_BOUND[0]) # Keep in bounds
    return individual

def genetic_algorithm():
    population = create_population(POPULATION_SIZE)
    best_individual = None
    best_fitness = float('-inf')

    for generation in range(GENERATIONS):
        fitnesses = evaluate_population(population)

        for i, fit in enumerate(fitnesses):
            if fit > best_fitness:
                best_fitness = fit
                best_individual = population[i]

        new_population = []
        while len(new_population) < POPULATION_SIZE:
            parent1, parent2 = select(population, fitnesses)
            child1, child2 = crossover(parent1, parent2)
            child1 = mutate(child1)
            child2 = mutate(child2)
            new_population.extend([child1, child2])

        population = new_population[:POPULATION_SIZE]

        print(f"Generation {generation+1}: Best Fitness = {best_fitness:.5f}, Best X = {best_individual:.5f}")

    print("\nOptimization complete!")
    print(f"Best solution: x = {best_individual:.5f}, f(x) = {best_fitness:.5f}")

if __name__ == "__main__":
    genetic_algorithm()

```

1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 2680, 26

```
Best solution found:
Fitness (X value): 20
```

Genetic Algorithm

Steps:

1. Define the problem
create a mathematical function to optimize
2. Initialize parameters.
population size, mutation rate, etc.,
3. create initial population,
generate initial population of potential solution
4. Evaluate fitness
Evaluate for each individual.
5. Selection
Based on fitness to reproduce
6. Crossover
perform crossover between individuals
7. mutation.
Repeat evaluation, selection, crossover,
mutation
8. Iteration : Repeat for n generation.
9. output the Best solution.

output generation Data

no	popn	value	fit	prob	%	expect	actual
1	12	001100	144	0.124	12.4	0.49	0
2	25	00011001	625	0.544	54.1	2.16	2
3	5	00101	25	0.021	2.16	0.03	1
4	19	10001	361	0.321	31.25	1.25	1

rating pool

no	rating pool	value	crosses	Fitness
1	11001	25	2	625
2	01001	25	2	625
3	10011	19	3	361
4	11001	25	3	625

mutation data

no	offspring	after mutation	fit	fitness
1	11000	-	25	625
2	11001	-	25	625
3	11001	-	25	625
4	10011	-	19	361

Fitness of overall best fit: 625
Best solution: 25

02: Particle Swarm Optimization for Function Optimization:

Algorithm :

Code:

```
import random
import math
def fitness_function(x):
    return x * math.sin(10 * math.pi * x) + 1

# 2. Initialize parameters
NUM_PARTICLES = 30
MAX_ITER = 100
W = 0.7
C1 = 1.5
C2 = 1.5
X_BOUND = (-1, 2)
V_MAX = 0.1

class Particle:
    def __init__(self):
        self.position = random.uniform(*X_BOUND)
        self.velocity = random.uniform(-V_MAX, V_MAX)
        self.best_position = self.position
        self.best_fitness = fitness_function(self.position)

    def update_velocity(self, global_best_position):
        r1 = random.random()
        r2 = random.random()
        cognitive = C1 * r1 * (self.best_position - self.position)
        social = C2 * r2 * (global_best_position - self.position)
        self.velocity = W * self.velocity + cognitive + social
        # Clamp velocity within limits
        self.velocity = max(min(self.velocity, V_MAX), -V_MAX)

    def update_position(self):
        self.position += self.velocity
        # Keep particle within bounds
        self.position = max(min(self.position, X_BOUND[1]), X_BOUND[0])

def particle_swarm_optimization():
    swarm = [Particle() for _ in range(NUM_PARTICLES)]

    global_best_particle = max(swarm, key=lambda p: p.best_fitness)
    global_best_position = global_best_particle.position
    global_best_fitness = global_best_particle.best_fitness

    for iteration in range(MAX_ITER):
        for particle in swarm:
```

```

current_fitness = fitness_function(particle.position)

if current_fitness > particle.best_fitness:
    particle.best_fitness = current_fitness
    particle.best_position = particle.position

# Update global best
if current_fitness > global_best_fitness:
    global_best_fitness = current_fitness
    global_best_position = particle.position

for particle in swarm:
    particle.update_velocity(global_best_position)
    particle.update_position()

    print(f"Iteration {iteration+1:03d} | Best Fitness = {global_best_fitness:.5f} | Best X = {global_best_position:.5f}")

print("\nOptimization complete!")
print(f"Best solution: x = {global_best_position:.5f}, f(x) = {global_best_fitness:.5f}")

if __name__ == "__main__":
    particle_swarm_optimization()

```

Output:

OUTPUT:

```

Iteration 1/20 - Best Fitness: 0.081095
Iteration 2/20 - Best Fitness: 0.081095
Iteration 3/20 - Best Fitness: 0.006725
Iteration 4/20 - Best Fitness: 0.003298
Iteration 5/20 - Best Fitness: 0.003298
Iteration 6/20 - Best Fitness: 0.003298
Iteration 7/20 - Best Fitness: 0.002956
Iteration 8/20 - Best Fitness: 0.002956
Iteration 9/20 - Best Fitness: 0.002956
Iteration 10/20 - Best Fitness: 0.002956
Iteration 11/20 - Best Fitness: 0.002956
Iteration 12/20 - Best Fitness: 0.001661
Iteration 13/20 - Best Fitness: 0.001066
Iteration 14/20 - Best Fitness: 0.001066
Iteration 15/20 - Best Fitness: 0.001066
Iteration 16/20 - Best Fitness: 0.000587
Iteration 17/20 - Best Fitness: 0.000587
Iteration 18/20 - Best Fitness: 0.000165
Iteration 19/20 - Best Fitness: 0.000095
Iteration 20/20 - Best Fitness: 0.000018

Best solution found:
Position: [ 0.00321441 -0.00268418]
Fitness: 1.7537291281392393e-05

```

12-9/25

Particle Swarm optimization

pseudocode

1. $P = \text{particle initialization } ()$
2. For $I = 1$ to max .
 3. for each particle P in P do.

$$fP = f(P)$$
 4. if fP is better than $f(P_{\text{best}})$

$$p_{\text{best}} = P$$
5. end
6. end
7. $g_{\text{best}} = \text{best } P \text{ is } T.$
8. for each particle P in P do
9.
$$V_i^{t+1} = \underbrace{V_i^t}_{\text{inertia}} + \underbrace{C_1 U_1^t (p_{\text{best}}^t - P_i^t)}_{\text{personal influence}} + \underbrace{C_2 U_2^t (g_{\text{best}}^t - P_i^t)}_{\text{social influence}}$$
10. $P_i^{t+1} = P_i^t + V_i^{t+1}$
11. end
12. end.

Example: $f(x, y) = x^2 + y^2$

initial = 0.3

value of cognitive + social constants

$$C_1 = 2 + C_2 = 2$$

Initial solⁿ are set to 1000

Initial P1 fitness value = $1^2 + 1^2 = 2$

particle no	Initial x, y	pos x, y	velocity x, y	Best sol ⁿ	Best x, y	fitness value
P1	1	1	0 0	1000	--	2
P2	-1	1	0 0	1000	--	2
P3	0.5	-0.5	0 0	1000	--	0.5
P4	1	1	0 0	1000	--	2
P5	0.25	0.25	0 0	1000	--	0.125

Iteration 2

Pno	Initial x, y	velocity x, y	Best sol ⁿ	Best pos x, y	fitness
P1	1 1	-0.75 -0.75	2	1 -1	2
P2	-1 1	1.25 -0.75	2	-1 1	2
P3	0.5 -0.5	-0.25 0.75	0.5	0.5 5	0.5
P4	1 -1	-0.75 1.25	2	1 -1	2
P5	0.25 0.25	0 0	0.125	0.25 0.25	0.125

Iteration 3

Pno	Initial x, y	velocity x, y	Best sol ⁿ	Best pos x, y	fitness
P1	0.25 0.25	-0.375 -0.375	2	1 1	0.125
P2	0.25 0.25	-0.625 -0.375	2	-1 1	0.125
P3	0.25 0.25	-0.125 0.375	0.5	0.5 0.5	0.125
P4	0.25 0.25	-0.875 0.625	2	1 -1	0.125
P5	0.25 0.25	0 0	0.125	0.25 0.25	0.125

Program 03: Ant Colony Optimization for the Traveling Salesman Problem:

Algorithm:

Code:

```
import numpy as np

def distance(city1, city2):
    return np.linalg.norm(city1 - city2)

def route_length(route, cities):
    dist = 0.0
    for i in range(len(route) - 1):
        dist += distance(cities[route[i]], cities[route[i+1]])
    dist += distance(cities[route[-1]], cities[route[0]])
    return dist

def select_next_city(current_city, unvisited, pheromone, heuristic, alpha, beta):
    pheromone_vals = pheromone[current_city, unvisited] ** alpha
    heuristic_vals = heuristic[current_city, unvisited] ** beta
    probs = pheromone_vals * heuristic_vals
    probs /= probs.sum()
    return np.random.choice(unvisited, p=probs)

def aco_tsp(cities, num_ants=20, num_iterations=20, alpha=1.0, beta=5.0, rho=0.5,
            initial_pheromone=1.0):
    num_cities = len(cities)

    pheromone = np.full((num_cities, num_cities), initial_pheromone)

    heuristic = np.zeros((num_cities, num_cities))
    for i in range(num_cities):
        for j in range(num_cities):
            if i != j:
                heuristic[i, j] = 1.0 / (distance(cities[i], cities[j]) + 1e-10)

    best_route = None
    best_length = float('inf')

    for iteration in range(num_iterations):
        all_routes = []
        all_lengths = []

        for ant in range(num_ants):
            route = []
            unvisited = list(range(num_cities))

            current_city = np.random.choice(unvisited)
            route.append(current_city)
```



```

        unvisited.remove(current_city)

        while unvisited:
            next_city = select_next_city(current_city, unvisited, pheromone,
            heuristic, alpha, beta)
            route.append(next_city)
            unvisited.remove(next_city)
            current_city = next_city

        length = route_length(route, cities)
        all_routes.append(route)
        all_lengths.append(length)

        if length < best_length:
            best_length = length
            best_route = route

    pheromone *= (1 - rho)

    for route, length in zip(all_routes, all_lengths):
        for i in range(num_cities - 1):
            pheromone[route[i], route[i+1]] += 1.0 / length
            pheromone[route[i+1], route[i]] += 1.0 / length

        pheromone[route[-1], route[0]] += 1.0 / length
        pheromone[route[0], route[-1]] += 1.0 / length

    print(f"Iteration {iteration+1}/{num_iterations} - Best Length:
    {best_length:.4f}")

    return best_route, best_length

if __name__ == "__main__":
    cities = np.array([
        [0, 0],
        [1, 5],
        [5, 2],
        [6, 6],
        [8, 3],
        [7, 9],
        [2, 8],
        [3, 3]
    ])

    best_route, best_length = aco_tsp(cities)

    print("\nBest route found:")
    print(best_route)
    print("Route length:", best_length)

```

Output:

```
Iteration 1/20 - Best Length: 31.6195
Iteration 2/20 - Best Length: 29.7691
Iteration 3/20 - Best Length: 29.7691
Iteration 4/20 - Best Length: 29.7691
Iteration 5/20 - Best Length: 29.7691
Iteration 6/20 - Best Length: 29.7691
Iteration 7/20 - Best Length: 29.7691
Iteration 8/20 - Best Length: 29.7691
Iteration 9/20 - Best Length: 29.7691
Iteration 10/20 - Best Length: 29.7691
Iteration 11/20 - Best Length: 29.7691
Iteration 12/20 - Best Length: 29.7691
Iteration 13/20 - Best Length: 29.7691
Iteration 14/20 - Best Length: 29.7691
Iteration 15/20 - Best Length: 29.7691
Iteration 16/20 - Best Length: 29.7691
Iteration 17/20 - Best Length: 29.7691
Iteration 18/20 - Best Length: 29.7691
Iteration 19/20 - Best Length: 29.7691
Iteration 20/20 - Best Length: 29.7691
```

Best route found:

```
[np.int64(3), np.int64(5), np.int64(6), np.int64(1), np.int64(0), np.int64(7), np.int64(2), np.int64(4)]
Route length: 29.76913194777377
```

class

Best solution : 2.5 Best : 28.2500

ant colony optimization

for the travelling sales man problem

Pseudo code

- ① Initialize pheromone levels τ on all edges
- ② set parameters : ants, alpha, (pheromone), beta, evaporation, rate, iteration
- ③ for iter = 1 to iteration do
 - for each ant in number of ants do
 - Initialize starting city random
 - while tour not complete do
 - choose next city based on probability
 - probability (pheromonic) * (heuristic * beta) / sum over all possible cities
 - move to chosen city and add it to the tour.
- end while

Calculate total length of the tour.
End for.

update pheromone on all edges:

Evaporate pheromone. $\text{pheromone} =$

$\cdot (1 - \text{evaporation_rate})$

$\cdot$ for each ant:

deposit pheromone proportional
quality on edges visited.

optionally, reinforce pheromone on
the globally best tour found so far.

End for.

Return the best tour found.

o/p:-

Best solution : $[3, 4, 2, 0, 1]$

Best length : 14

For Ret all

Program 04: Cuckoo Search (CS):

Algorithm;

Code:

```
import numpy as np
import math

def food_availability(x):
    return - (x - 5) ** 2 + 20

def levy_flight(Lambda=1.5):
    sigma_u = (math.gamma(1 + Lambda) * math.sin(math.pi * Lambda / 2) /
               (math.gamma((1 + Lambda) / 2) * Lambda * 2 ** ((Lambda - 1) / 2)))
    ** (1 / Lambda)
    sigma_v = 1
    u = np.random.normal(0, sigma_u)
    v = np.random.normal(0, sigma_v)
    step = u / (abs(v) ** (1 / Lambda))
    return step

def cuckoo_search_nest_location(n=10, iterations=200, pa=0.25, lower_bound=0,
                                upper_bound=10):
    nests = np.random.uniform(lower_bound, upper_bound, n)
    fitness = np.array([food_availability(x) for x in nests])

    best_idx = np.argmax(fitness)
    best_nest = nests[best_idx]
    best_fitness = fitness[best_idx]

    Lambda = 1.5

    for t in range(iterations):
        for i in range(n):
            step_size = 0.1 * levy_flight(Lambda)
            new_nest = nests[i] + step_size * (nests[i] - best_nest)
            new_nest = np.clip(new_nest, lower_bound, upper_bound)

            new_fitness = food_availability(new_nest)
            if new_fitness > fitness[i]:
                nests[i] = new_nest
                fitness[i] = new_fitness

            if new_fitness > best_fitness:
                best_fitness = new_fitness
                best_nest = new_nest

        num_abandon = int(pa * n)
        worst_indices = np.argsort(fitness)[:num_abandon]
        nests[worst_indices] = np.random.uniform(lower_bound, upper_bound,
                                                    num_abandon)
        fitness[worst_indices] = [food_availability(x) for x in
                                   nests[worst_indices]]
```



```

    current_best_idx = np.argmax(fitness)
    if fitness[current_best_idx] > best_fitness:
        best_fitness = fitness[current_best_idx]
        best_nest = nests[current_best_idx]

        if (t+1) % 20 == 0 or t == 0:
            print(f"Iteration {t+1}: Best nest location = {best_nest:.4f}, Max food
= {best_fitness:.6f}")

    return best_nest, best_fitness

if __name__ == "__main__":
    best_location, max_food = cuckoo_search_nest_location()
    print("\nFinal Results:")
    print(f"Best nest location: {best_location:.4f}")
    print(f"Maximum food collected: {max_food:.6f}")

```

Output:

```

Iteration 1: Best nest location = 5.3102, Max food = 19.903783
Iteration 20: Best nest location = 4.9646, Max food = 19.998744
Iteration 40: Best nest location = 4.9646, Max food = 19.998744
Iteration 60: Best nest location = 5.0038, Max food = 19.999985
Iteration 80: Best nest location = 4.9992, Max food = 19.999999
Iteration 100: Best nest location = 5.0000, Max food = 20.000000
Iteration 120: Best nest location = 5.0000, Max food = 20.000000
Iteration 140: Best nest location = 5.0000, Max food = 20.000000
Iteration 160: Best nest location = 5.0000, Max food = 20.000000
Iteration 180: Best nest location = 5.0000, Max food = 20.000000
Iteration 200: Best nest location = 5.0000, Max food = 20.000000

```

Final Results:

Best nest location: 5.0000

Maximum food collected: 20.000000

Lab-5

cuckoo Search Algorithm

pseudocode

Start cuckoo Search Algorithm

① set parameters -

- n = number of sets (solutions)
- pa = chance of discovering a bad egg
e.g., 0.25
- MAX gen = number of iterations
(generations)

② create n random solutions (nests)

③ Repeat for MAX gen times:

a. for each cuckoo (solution):

i) make a new solution using a
random jump

ii) if the new solution is better than
the old one:

keep the new one (replace the old)

b. for each nest:

i. with a small chance (pa), forget
the bad nests

ii) replace them with new random solution

c. remember the best solution found so far

(4) after all generations, return the best solution

End algorithm

o/p:

Gen 1: Best $x = -0.42866$, $f(x) = 11.78999$

Gen 5: Best $x = 5.50779$, $f(x) = 6.78899$

Gen 10: Best $x = 6.63823$, $f(x) = 15.28640$

Gen 15: Best $x = 9.26885$, $f(x) = 39.29647$

Gen 20: Best $x = 6.49781$, $f(x) = 10.28668$

Gen 25: Best $x = -1.44701$, $f(x) = 19.34508$

Gen 30: Best $x = -0.21804$, $f(x) = 10.84808$

Gen 35: Best $x = 9.11414$, $f(x) = 37.8872$

Gen 40: Best $x = 0.76955$, $f(x) = 4.97492$

Gen 45: Best $x = -5.08460$, $f(x) = 65.40931$

Gen 50: Best $x = -3.3412$, $f(x) = 40.12114$

optimization finished!

Best solution found: $x = -3.33412$, $f(x) = 40.12114$

Program 05: Grey Wolf Optimizer (GWO):

Algorithm:

Code:

```
import numpy as np

def nectar_availability(position):
    x, y = position
    term1 = 20 * np.exp(-((x - 3) ** 2 + (y - 3) ** 2) / 4)
    term2 = 15 * np.exp(-((x - 7) ** 2 + (y - 7) ** 2) / 3)
    term3 = 10 * np.exp(-((x - 5) ** 2 + (y - 8) ** 2) / 2)
    return term1 + term2 + term3

def gwo_beehive(n_wolves=30, iterations=20, lower_bound=0, upper_bound=10):
    dim = 2

    wolves = np.random.uniform(lower_bound, upper_bound, (n_wolves, dim))

    alpha_pos = np.zeros(dim)
    alpha_score = -np.inf
    beta_pos = np.zeros(dim)
    beta_score = -np.inf
    delta_pos = np.zeros(dim)
    delta_score = -np.inf

    for t in range(iterations):
        for i in range(n_wolves):
            fitness = nectar_availability(wolves[i])

            if fitness > alpha_score:
                alpha_score = fitness
                alpha_pos = wolves[i].copy()
            elif fitness > beta_score:
                beta_score = fitness
                beta_pos = wolves[i].copy()
            elif fitness > delta_score:
                delta_score = fitness
                delta_pos = wolves[i].copy()

        a = 2 - t * (2 / iterations)

        for i in range(n_wolves):
            for j in range(dim):
                r1 = np.random.rand()
                r2 = np.random.rand()
                A1 = 2 * a * r1 - a
                C1 = 2 * r2
                D_alpha = abs(C1 * alpha_pos[j] - wolves[i][j])
                X1 = alpha_pos[j] - A1 * D_alpha

                r1 = np.random.rand()
                r2 = np.random.rand()
                A2 = 2 * a * r1 - a
                C2 = 2 * r2
                D_beta = abs(C2 * beta_pos[j] - wolves[i][j])
                X2 = beta_pos[j] - A2 * D_beta
```



```

        r1 = np.random.rand()
        r2 = np.random.rand()
        A3 = 2 * a * r1 - a
        C3 = 2 * r2
        D_delta = abs(C3 * delta_pos[j] - wolves[i][j])
        X3 = delta_pos[j] - A3 * D_delta

        wolves[i][j] = (X1 + X2 + X3) / 3

    wolves[i] = np.clip(wolves[i], lower_bound, upper_bound)

    print(f"Iteration {t+1:2d}: Best nectar = {alpha_score:.6f} at location
x={alpha_pos[0]:.4f}, y={alpha_pos[1]:.4f}")

    return alpha_pos, alpha_score

if __name__ == "__main__":
    best_hive_location, max_nectar = gwo_beehive()
    print("\nFinal optimal beehive location:")
    print(f"x = {best_hive_location[0]:.4f}, y = {best_hive_location[1]:.4f}")
    print(f"Maximum nectar availability: {max_nectar:.6f}")

```

Output:

```

Iteration 1: Best nectar = 14.314430 at location x=5.9729, y=6.9847
Iteration 2: Best nectar = 14.314430 at location x=5.9729, y=6.9847
Iteration 3: Best nectar = 18.833972 at location x=3.4183, y=2.7443
Iteration 4: Best nectar = 18.833972 at location x=3.4183, y=2.7443
Iteration 5: Best nectar = 19.918544 at location x=2.8770, y=3.0355
Iteration 6: Best nectar = 19.918544 at location x=2.8770, y=3.0355
Iteration 7: Best nectar = 19.918544 at location x=2.8770, y=3.0355
Iteration 8: Best nectar = 19.918544 at location x=2.8770, y=3.0355
Iteration 9: Best nectar = 19.926097 at location x=3.1064, y=2.9403
Iteration 10: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 11: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 12: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 13: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 14: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 15: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 16: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 17: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 18: Best nectar = 19.976975 at location x=3.0522, y=3.0444
Iteration 19: Best nectar = 19.990873 at location x=2.9582, y=3.0120
Iteration 20: Best nectar = 19.999439 at location x=3.0134, y=2.9978

Final optimal beehive location:
x = 3.0134, y = 2.9978
Maximum nectar availability: 19.999439

```

grey wolf optimization

applications

- * Engineering design and optimization
- * machine learning hyperparameter tuning

pseudocode

- * Define vector function (x, y) as sum of gaussian clusters
- * initial wolf population with random positions
- * Evaluate fitness of wolves (vector initial (x, y))
- * Identify alpha, beta, delta wolves by beta fitness.

For each iteration:

update parameter 'a' decreasing from 2 to 10.

for each wolf.

update position influenced by alpha, beta, delta wolves clamp position within field boundaries.

- * Evaluate fitness and update alpha, beta, delta wolves
- * Return alpha wolf position and fitness as best fit location and vector amount

optical lattice location: $x = 3.0015$

$y = 1.0920$ position vector available

$z = 10.000000$

Program 6: Parallel Cellular Algorithms and Programs:

Algorithm:

Code:

```
import numpy as np

def rosenbrock(x):
    return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2

grid_size = (10, 10)
num_dimensions = 2
num_iterations = 200
beta0 = 1.0          # Base attractiveness
gamma = 1.0          # Light absorption coefficient
alpha = 0.2          # Randomness factor
radius = 1
search_bounds = (-2, 2)

cells = np.random.uniform(search_bounds[0], search_bounds[1],
                           size=(grid_size[0], grid_size[1], num_dimensions))

get neighbors (Moore neighborhood)
def get_neighbors(cells, i, j):
    neighbors = []
    for di in range(-radius, radius + 1):
        for dj in range(-radius, radius + 1):
            if di == 0 and dj == 0:
                continue
            ni, nj = (i + di) % grid_size[0], (j + dj) % grid_size[1]
            neighbors.append(cells[ni, nj])
    return np.array(neighbors)

best_solution = None
best_fitness = float('inf')

for t in range(num_iterations):
    new_cells = np.copy(cells)

    for i in range(grid_size[0]):
        for j in range(grid_size[1]):
            firefly = cells[i, j]
            fit_i = rosenbrock(firefly)
            neighbors = get_neighbors(cells, i, j)

            for n in neighbors:
                fit_n = rosenbrock(n)
                if fit_n < fit_i: # brighter (better)
                    distance = np.linalg.norm(firefly - n)
                    beta = beta0 * np.exp(-gamma * distance**2)
                    step = beta * (n - firefly) + alpha * np.random.uniform(-1, 1,
num_dimensions)
                    firefly = firefly + step
                    fit_i = rosenbrock(firefly)

            new_cells[i, j] = firefly

    if fit_i < best_fitness:
```



```

        best_fitness = fit_i
        best_solution = firefly

    cells = new_cells

    if t % 20 == 0:
        print(f"Iteration {t}: Best Fitness = {best_fitness:.6f}")

print("\n=====")
print(" Parallel Cellular Firefly Algorithm Results")
print("=====")
print("Best Solution:", best_solution)
print("Best Fitness:", best_fitness)

```

Output:

```

Iteration 0: Best Fitness = 0.120322
Iteration 20: Best Fitness = 0.000162
Iteration 40: Best Fitness = 0.000114
Iteration 60: Best Fitness = 0.000049
Iteration 80: Best Fitness = 0.000049
Iteration 100: Best Fitness = 0.000049
Iteration 120: Best Fitness = 0.000049
Iteration 140: Best Fitness = 0.000049
Iteration 160: Best Fitness = 0.000009
Iteration 180: Best Fitness = 0.000009

```

```
=====
```



Parallel Cellular Firefly Algorithm Results

```
=====
```

```
Best Solution: [0.99738409 0.99491349]
```

```
Best Fitness: 8.760504339778715e-06
```

parallel cellular algorithm

algo:

initialization

input: gridsize ($m \times n$)

input: max iterations

initialize cell $[m][n]$ with initial states

define neighborhood

define transition Rule (cell, neighbors)

for $i = 1$ to max iterations do

parallel for each cell $[i, j]$ in cell do

neighbors $\leftarrow$ getNeighbors (cell, i, j)

newcell $[i][j] \leftarrow$ transition Rule cell $[i][j]$, neighbors)

end parallel for

synchronize()

parallel for each cell (i, j) in cell do

cell $[i][j] \leftarrow$ newcell $[i][j]$

end parallel for

synchronize()

end for

output: final state of cell $[m][n]$

p/p :

Iteration 0 : Best fitness = 6.113738

Iteration 20 : Best fitness = 0.000406

Iteration 40 : Best fitness = 0.000145

Iteration 180 : Best fitness = 0.00008

Best solution : [1.00253 1.00517]

Best fitness : 7.95067

Sen
11/11

Program 7: Optimization via Gene Expression Algorithms:

Algorithm:

Code:

```
import random
import math

def fitness_function(x):
    return x * math.sin(10 * math.pi * x) + 1 # Objective: maximize this

POPULATION_SIZE = 30
NUM_GENES = 10      # Number of genes per chromosome
MUTATION_RATE = 0.1
CROSSOVER_RATE = 0.8
GENERATIONS = 100
X_BOUND = (-1, 2)

def create_individual():
    return [random.uniform(X_BOUND[0], X_BOUND[1]) for _ in range(NUM_GENES)]

def create_population(size):
    return [create_individual() for _ in range(size)]

def express_genes(individual):
    # Expression step: translate gene sequence into a single functional value (x)
    expressed_x = sum(individual) / len(individual)
    return expressed_x

def evaluate_fitness(individual):
    x = express_genes(individual)
    return fitness_function(x)

def tournament_selection(population, k=3):
    selected = random.sample(population, k)
    selected.sort(key=lambda ind: evaluate_fitness(ind), reverse=True)
    return selected[0]

def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, NUM_GENES - 1)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]
        return child1, child2
    else:
        return parent1[:], parent2[:]

def mutate(individual):
    for i in range(NUM_GENES):
        if random.random() < MUTATION_RATE:
            individual[i] += random.uniform(-0.1, 0.1)
            individual[i] = max(min(individual[i], X_BOUND[1]), X_BOUND[0])
    return individual
```



```

def gene_expression_algorithm():
    population = create_population(POPULATION_SIZE)
    best_individual = None
    best_fitness = float('-inf')

    for generation in range(GENERATIONS):
        new_population = []

        for ind in population:
            fit = evaluate_fitness(ind)
            if fit > best_fitness:
                best_fitness = fit
                best_individual = ind

        while len(new_population) < POPULATION_SIZE:
            parent1 = tournament_selection(population)
            parent2 = tournament_selection(population)
            child1, child2 = crossover(parent1, parent2)
            child1 = mutate(child1)
            child2 = mutate(child2)
            new_population.extend([child1, child2])

        population = new_population[:POPULATION_SIZE]

        expressed_x = express_genes(best_individual)
        print(f"Generation {generation+1:03d} | Best Fitness = {best_fitness:.5f} | Best X = {expressed_x:.5f}")

        final_x = express_genes(best_individual)
        print("\nOptimization complete!")
        print(f"Best solution: x = {final_x:.5f}, f(x) = {best_fitness:.5f}")

if __name__ == "__main__":
    gene_expression_algorithm()

```

Output:

```
Output
Generation 1: Best Value = 6.363236
Generation 2: Best Value = 6.363236
Generation 3: Best Value = 6.363236
Generation 4: Best Value = 6.363236
Generation 5: Best Value = 4.087452
Generation 6: Best Value = 3.420658
Generation 7: Best Value = 3.420658
Generation 8: Best Value = 3.420658
Generation 9: Best Value = 3.420658
Generation 10: Best Value = 3.420658

Best Solution Found: [0.07026351598497271, -0.36887618913241305, 0
.16261776749364998, 1.729095856868689, 0.513258486361484]
Best Value: 3.4206578989545884
```

Lab

Gene Expression Algorithm

- Initialization.
- Fitness Assignment
- Selection.
- Crossover
- Mutation
- Gene Expression
- Termination.

Steps: Fitness (x) = x^2 .

Select encoding techniques.

Use with terminals (variables, constant) and function (+, -, $\div$).

2) Initial population.

S.No.	Initial	phenotype	value	Fitness	P.
1	+ x x	x^2	12	144	0.1747
2	+ x x	$2x$	25	625	0.841
3	x	x	5	25	0.0236
4	- x ²	$x - 2$	19	361	0.3125

$$\sum P(x) = 1.155$$

$$\text{Avg} = 288.75$$

Actual	Expected count
1	0.5
2	2.1
0	0.08
1	1.25

2)

S.No	Selection	Selected	% of pop	offspring	phenomena
1		+ x x	2	+ x +	x^2 ()
2		+ x x	1	+ x x	$2x$
3		+ x x	3	+ x x	$2x$
2		- x x	1	+ x -	x

μ value	fitness
1.5	169
2.4	576
2.7	729
1.7	289

c) crossover
perform crossovers at randomly chosen
gene locations
max fitness = 729

5. mutation.

S No	offspring	mutation	offspring	prootype
1	+ x +	+ $\rightarrow$ -	+ μ -	$x * (x -)$
2	+ x μ	None	+ x μ	2 μ
3	+ μ -	- $\rightarrow$ +	+ x +	$x + \mu * \mu$
4	+ μ 0	None	+ μ 2	$\mu + 2$

μ value	fitness
1.29	841
2.4	576
2.7	729
2.0	400

6) gene expression and evolution Decode.
Each genotype $\rightarrow$ phenotype.

$$\Sigma f(x) = 841 + 576 + 729 + 400$$

$$= 2546$$

$$\text{Avg} = 636.5$$

$$\text{max} = 841$$

7. Iterate until convergence.
Repeat step 3-6 until fitness improvement
converges to 0.

Pseudo code:

- Define fitness function.
- Define parameters.
- Create population.
- Select mating pool.
- Mutation after mating.
- Gene expression.
- Iteration.
- Output best value.

Output:

$x = 26.37$
 $f(x) = 695.45$